

Microservice

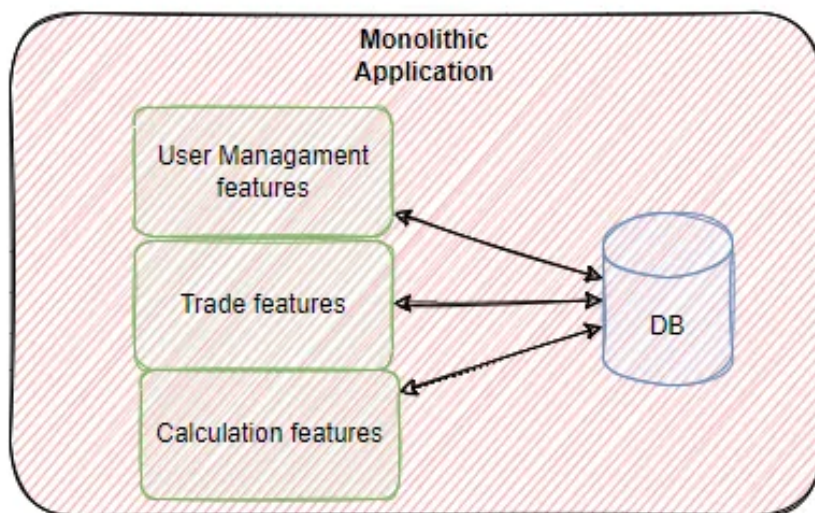
Monday, 17 August 2026

10:17 pm

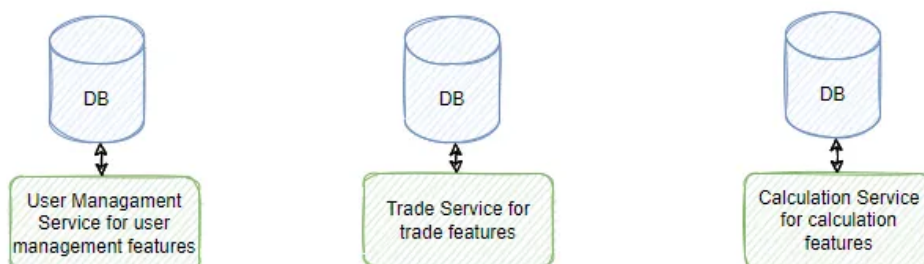
- Microservices are like individual workers in a big company. Each worker has a specific job to do, and they can do their job without needing constant supervision or help from others. They also have their own tools and equipment to get their work done. In the same way, microservices are small, self-contained pieces of software in a larger computer program. They each perform a specific task, and you can update, check on, or put them into action separately without affecting the whole program
- Microservices architecture is used in large and complex organisations where a number of development teams can work independently to deliver business functionality and where applications are required to serve a business domain
- Microservice is based on SOA (Service-Oriented Architecture) // TODO need to research more about this architecture

Microservice vs Monolithic

This example is about a monolithic application. 3 features share the same common DB.



Then we create a microservice for each business feature. For the example above, we now have three services (see below) and each service has its own native database. Now 3 services can communicate via RestAPI or through a messaging router



Benefits of Microservices

- Each service can be hosted and deployed independently of the others. This is beneficial for testing and deployment. It also helps prevent the failure of one service from affecting the

testing and deployment. It also helps prevent the failure of one service from affecting the entire application.

- Can choose different tech stack for each service
- All services can communicate with each other even the services live on different machines.
- Easier to scale up if there is a service having bottleneck, we can move them to another machine without affecting other services
- Code re-usability, a feature is hosted as a service and it allows service to consume the same feature without implementing the same code twice

Disadvantages

- Increase the cost for maintenance
- Organizational complexity and overhead. Because each service may have the different tech stack, it is hard to manage in the long term and by whom